

# Design and Evaluation of a Multimodal ReAct Agent: A Qwen3/Qwen3-VL Course Project

Tian Huang<sup>a</sup>, Qixuan Li<sup>b</sup>, and Ziun Mo<sup>c</sup>

<sup>a</sup>AIAA3102 group 10, HKUST(GZ)

This manuscript was compiled on December 7, 2025

For the final project of “Python Programming for AI,” we designed and implemented a comprehensive multimodal ReAct agent system. The primary objective was to build an architecture that remains performant under limited computational resources while integrating complex capabilities such as multi-turn reasoning, external tool execution, and visual understanding. Centered on the Qwen3/Qwen3-VL model family, the system is architected to serve both a local Command Line Interface (CLI) and a modern FastAPI + Vite Web UI, ensuring versatility for different user needs. It seamlessly integrates a retrieval-augmented generation (RAG) pipeline, a suite of functional tools, and a proactive safety mechanism. We construct a Wiki-based vector store and RAG engine, expose Google/Tavily search, math, and environment tools through a unified toolkit, and design a heuristic PromptGuard module to defend against prompt injection and tool abuse. Experiments cover ALFWorld embodied interaction, RAG QA, Web search augmentation, and safety evaluation, plus ablations on ReAct loops, RAG, and Web search. Compared to a non-ReAct baseline, the multimodal ReAct agent improves ALFWorld task success by 37 percentage points and action success by 20 points; RAG and live search substantially boost answer quality but roughly double latency; PromptGuard reaches 80% detection with 0 false positives on an extended attack set. Overall, we demonstrate the practicality and effectiveness of unifying ReAct, tools, RAG, and safety under engineering constraints, and surface future directions on latency, hallucination control, and semantic safety.

Multimodal Agent | ReAct | Retrieval-Augmented Generation (RAG) | Web Search | Safety

This work targets the final project of “Python Programming for AI.” Our goal is to deliver a multimodal ReAct agent that is runnable, measurable, and protected under tight compute budgets. The system centers on Qwen3/Qwen3-VL, serves both CLI and FastAPI + Web UI, and integrates multi-turn reasoning, tool use, RAG, and safety. Key challenges addressed in this project include: (1) Stability in Reasoning Loops: Maintaining a coherent Thought → Action → Observation loop within a strict step budget is difficult, particularly when the model must strictly adhere to specific JSON formats for tool invocation; (2) Multimodal Context Management: Effectively carrying image data and conversational context across multiple turns without exhausting the context window or causing GPU memory overflows; and (3) Security in Open Deployments: Defending against prompt injection, sensitive data leakage, and potential tool abuse is critical when exposing an agent to untrusted user inputs. Our contributions are: (1) a modular ReAct agent architecture that decouples reasoning, tool execution, and safety checks, allowing for rapid deployment on course servers; (2) a unified entry-point design where CLI, FastAPI, and Vite front-end pathways share the same underlying logic; (3) a Wiki-based vector store and RAG engine; (4) a PromptGuard module inspired by LLM\_OWASP\_Scanner with tests for its core

rules; and (5) reproducible benchmarks for ALFWorld, RAG, Web search, and safety.

## 1. System Design

**A. Architecture Overview.** We follow a “multimodal ReAct agent + toolkit + safety layer + multi-end entry” pattern: user input from CLI or Web flows through FastAPI or the local process to the Agent, which relies on `ToolsManager` to dispatch search, math, RAG, and environment tools, optionally reading `rag/wiki_vector_db`. Each session keeps its own message, image, and tool-call history so the agent can reason over past Observations while respecting a strict step budget. The safety module intercepts inputs before tool execution, blocks high-risk content, and annotates the session state with lock flags. Logging and tracing capture every Thought/Action/Observation triple and tool latency for later analysis, while a simple persistence layer writes chat and tool events to disk for reproducibility. Figure 1 shows the end-to-end path and how the safety and RAG branches sit alongside the LLM core.

## B. Core Modules.

- `main.py`: CLI entry with image commands, PromptGuard reset, logging, optional RAG loading, and lightweight session scaffolding for local runs.
- `agent.py`: ReAct loop orchestration, tool spec assembly, format checks, tool invocation, and history management; also emits corrective system prompts when the LLM drifts from the Thought/Action/Observation schema.

### Significance Statement

This paper presents the design and evaluation of a practical multimodal ReAct agent system built for an advanced AI applications course. The system integrates Qwen3 and Qwen3-VL models with a unified tool framework, retrieval-augmented generation, and a safety module, and is accessible via CLI, FastAPI, and a web UI. Through benchmarks on ALFWorld, RAG-based QA, web search, and prompt injection tests, we provide reproducible evidence that combining ReAct reasoning, external tools, and retrieval can substantially improve task success and robustness, while also highlighting the trade-offs in latency and remaining safety challenges.

All authors jointly designed the system architecture and experiments; implementation of the ReAct agent and tool framework was led by the first author; RAG pipeline and analysis scripts were led by the second author; the third author focused on the safety module and web interface. All authors contributed to writing and revising the manuscript.

The authors declare no competing interests.

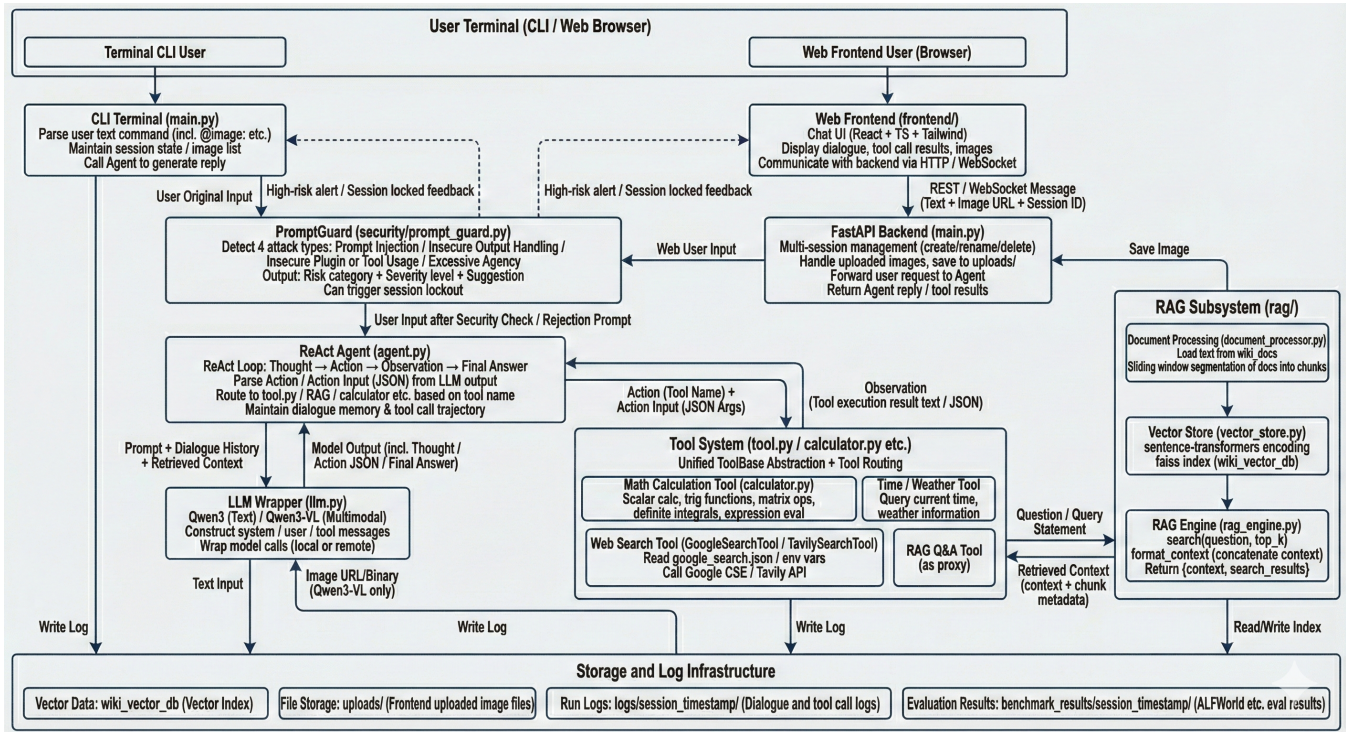


Fig. 1. Overall system: multi-end entry → Agent + ToolsManager → LLM (Qwen3/Qwen3-VL) → RAG/external tools/safety.

- **llm.py**: BaseLLM abstraction with Qwen3 (text) and Qwen3VL (multimodal) implementations, unified chat APIs, streaming hooks, and image attach/strip helpers for memory control. Since visual tokens consume significant context length, our implementation strictly limits the history of processed images. The agent retains full visual context for the current turn but summarizes or discards pixel data from previous turns, relying instead on the text-based "Observation" history to maintain continuity. This strategy allows the agent to run on consumer-grade GPUs (e.g., RTX 3090) while preventing Out-Of-Memory (OOM) errors during extended interactions.
- **tool.py + calculator.py**: tool signatures, parameter validation, timeout/error wrapping, and invocation for search, time/weather, math, RAG query, and system tools; outputs are normalized for prompt alignment.
- **rag/**: Contains `document_processor.py`, `vector_store.py`, and `rag_engine.py`. These modules handle the end-to-end retrieval pipeline, including sliding-window text chunking to preserve context, embedding generation using sentence-transformers, FAISS-based vector indexing (L2 distance), and a top-k stitching logic that filters irrelevant chunks based on score thresholds before prompt injection.
- **security/prompt\_guard.py**: Implements a heuristic detection engine inspired by LLM\_OWASP\_Scanner. It identifies potential threats by matching input patterns against a set of predefined regular expressions covering varying risk levels. It exposes structured findings (risk ratings and categories) consumed by both the CLI and backend to enforce immediate request blocking.

- **backend/main.py**: FastAPI multi-session service for chat, tool lists, image upload, static files, and WebSocket streaming, with per-session logs and guard status surfaced to clients.
- **frontend/**: Vite + TS + Tailwind UI with session list, message bubbles, image preview, LaTeX rendering, tool log view, and guard warnings; mirrors the backend JSON protocol for parity with CLI.

**C. Design Principles.** We emphasize lightweight code (few readable files for quick debugging), replaceability (clear interfaces between LLM, tools, RAG, safety), runtime robustness (logging, format checks, actionable errors, Guard locks), and entry consistency (CLI and Web share the same agent and tools). Every component can be swapped by editing a single factory or path (LLM choice, vector DB path, search provider). Observability is built-in via structured logs, per-tool latency, and safety flags so failure cases can be reproduced. Defensive defaults—timeouts, retries, and strict format validation—aim to keep the loop stable on limited GPUs and noisy prompts. The front end, backend, and CLI deliberately reuse the same JSON payloads and tool schemas to avoid configuration drift across entrypoints.

## 2. Key Design Choices

**A. ReAct Prompt and Agent.** We use bilingual system prompts with hard constraints (REACT\_PROMPT) listing ten rules that stress “one tool per step,” “facts must come from Observations,” and “separate chit-chat from factual queries.” The prompt also reminds the model to summarize briefly before acting, avoid hallucinating APIs, and stop early when confidence is low. `Agent.check_response` counts occurrences of

Thought/Action/Final Answer and issues corrective system prompts when formats drift. This mechanism is crucial for smaller models like Qwen3-8B, which may occasionally forget the strict JSON format required for tool calls after long context windows. By intercepting malformed outputs and injecting a specific error message asking for a retry in the correct format, we significantly increase the robustness of the reasoning loop without user intervention. `parse_latest_plugin_call` executes only the most recent Action and warns about multiples to prevent unsafe concurrent tool calls. `call_plugin` parses with `json5` to tolerate single quotes or trailing commas and returns structured errors that are fed back as Observations. `ToolsManager` keeps I/O templates consistent and injects human-readable descriptions so both users and the LLM align on tool semantics. For multimodal turns, only the first round sends the image while later turns rely on history to avoid redundant transfer and to align with Qwen3-VL context caching; when the context window tightens, the agent trims older Thoughts and keeps the latest Observations.

**B. Choosing Qwen3-VL.** We pick lightweight Qwen3-VL (e.g., 8B) to fit single RTX 3090/4090 GPUs while retaining image understanding for page parsing, screenshot QA, and ALFWorld scenes. The choice balances latency, memory, and multimodal accuracy, and the code keeps the model pluggable so a larger checkpoint or a hosted endpoint can be dropped in via config. The CLI offers `@image`, `@paste`, `@show`, `@clear`; the front end supports upload and URL images over WebSocket, with size checks and downscale to avoid context bloat. `llm.py` exposes `supports_multimodal` so the agent can gate image use and fall back to text-only when resources are tight. A text-only Qwen3 path remains for constrained resources, and all sampling parameters (temperature, `top_p`, `max_tokens`) are centralized for quick tuning under different hardware budgets.

**C. Tool System.** All tools share a `TOOL_DESC` template that serves both the LLM prompt and the CLI/Web tool list for human-LLM alignment. `tool.py` groups external search/time/weather, math (`calculator.py`), RAG retrieval, and system utilities, and normalizes outputs into concise Observations. `ToolsManager` reads `rag_db_path` at init to auto-enable or disable RAG tools, and injects per-tool examples to reduce misuse. Action inputs are parsed with `json5`; multiple actions trigger warnings; execution errors emit system hints so users can adjust parameters, reducing crashes in ALFWorld and Web benchmarks. Timeouts, retries, and coarse circuit-breakers protect slow Web tools; cached search results are reused within a session when queries repeat.

**D. RAG Pipeline.** `document_processor.py` slides over `wiki_docs/` AI/ML articles (window 512, stride 256, configurable) while preserving headings and section order; it drops blank lines, trims whitespace, and stores metadata for later citation. `vector_store.py` uses sentence-transformers embeddings with FAISS (L2) and persists both index and metadata so rebuilds are incremental. `rag/build_vector_db.py` accepts configurable paths for expansion or replacement, enabling quick swaps of corpus or embedding model. `rag_engine.py` exposes `retrieve` and `format_context` to control top-k, score thresholds, and context stitching before injecting into the LLM prompt; it also de-duplicates near-identical chunks and truncates context to fit the prompt budget.

**E. PromptGuard Safety Layer.** `security/prompt_guard.py` distills four high-risk classes from `LLM_OWASP_Scanner`—Prompt Injection, Insecure Output Handling, Insecure Plugin/Tool Usage, Excessive Agency—with multilingual regexes. Unicode normalization, zero-width cleanup, and whitespace folding yield a `normalized_prompt`; matches return a `PromptGuardFinding`, and `has_high_risk` blocks severity=High. `agent.text` invokes `_apply_prompt_guard` after user input; when locked, it returns a warning and marks the session until CLI `reset guard` is issued, preventing any tool calls while locked. `tests/test_prompt_guard.py` and `test_agent_prompt_guard.py` cover core cases, including multilingual injections, tool-abuse attempts, and benign prompts to keep false positives low. Guard outcomes are echoed to both CLI and Web UI so users understand why requests are blocked.

**F. API and Front End.** FastAPI offers multi-session CRUD, chat, image upload, tool lists, and WebSocket streaming, with `loguru` tracing tool calls and guard decisions. Sessions persist chat, tool traces, and guard status; the backend enforces per-session rate limits and rejects unsafe requests flagged by PromptGuard. The Vite + TS + Tailwind front end provides session lists, message bubbles, image preview, LaTeX rendering, tool logs, and guard banners, sharing the same JSON protocol as the backend so behaviors stay consistent across endpoints. A small debug pane exposes raw Observations and Actions for reproducibility when comparing CLI vs. Web behavior.

### 3. Experimental Setup and Evaluation

**A. ReAct / Embodied Interaction (ALFWorld).** `benchmarks/React/ALFworld.py` runs the 20 official tasks (place/heat/clean, etc.) with a 40-step cap. We execute both “React” and “React\_baseline” variants, logging to `detailed_results.csv/json`. Metrics: task success, average steps/success steps, action success; we parse `task_preview` and `error` for failure causes, with `analysis/analyze_react.py` producing Markdown and charts.

**B. RAG Evaluation.** We build QA over `wiki_docs`, comparing “LLM only” vs. “RAG enabled.” Scoring uses a larger deepseek3.2 judge and cross-checks historical scores for consistency.

**C. Web Search + RAG Evaluation.** Using `GoogleSearchTool/TavilySearchTool` plus `rag_engine.py`, we test factual QA, timeliness (news/events), and multi-page synthesis. We compare “LLM only” vs. “Web search enabled,” measuring logical correctness and factual accuracy (deepseek3.2 and human review).

**D. Safety / Prompt Injection Evaluation.** Based on `test_prompt_guard.py` and `test_agent_prompt_guard.py`, attack cases cover system override, leakage, disabling safeguards, and tool misuse. We report detection and false-positive rates.

## 4. Results and Error Analysis

**A. ALFWorld Results.** `analysis/analyze_react.py` summarizes `benchmark_results/React` and `React_baseline` into `react_analysis.md`: React finishes 17/22 tasks (77.27% success), average 17.95 steps, 83.03% action success; baseline 8/20 (40.00%), 27.10 steps, 63.00% action success. Compared to the baseline, React issues fewer redundant moves, recovers after bad observations, and uses shorter tool traces, indicating stronger coupling between Thought and Observation. React’s five failures stem from decoder prompt limits or mid-trajectory truncation; we plan chain truncation, higher `max_new_tokens`, or compressing older Thoughts. Baseline often wanders on examine/clean/heat mixes with <30% action accuracy due to weak grounding and missing “done” checks; adding affordance hints or simple action priors could lift it without full ReAct.

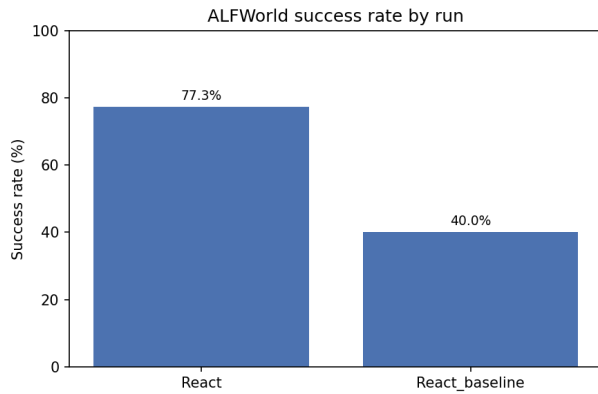


Fig. 2. React vs. Baseline success rates on ALFWorld.

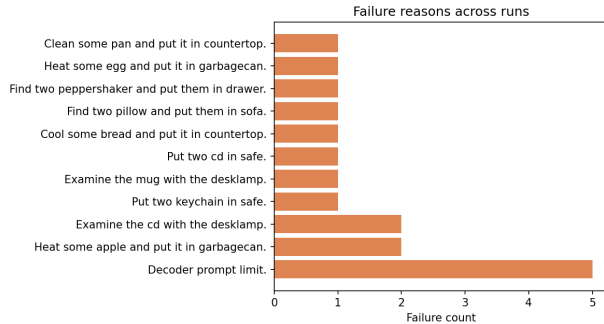


Fig. 3. Failure reasons under React (dominated by decoder prompt limit).

**B. RAG Results.** Across 20 samples, `benchmark_results/RAG` vs. `RAG_baseline` yields mean 0.16 vs. 0.07 (+0.09), median 0.12 vs. 0.06, and average latency 21.6s vs. 10.9s. Subsets: `RAG+calc` 0.20 vs. 0.08 (+0.12); `RAG test` 0.12 vs. 0.05 (+0.07). Retrieval + computation clearly improves scores but roughly doubles latency; low scores cluster when retrieval or calculation is missing. Regarding latency, the RAG pipeline roughly doubles the response time compared to the baseline. This increase is attributed to two main factors: the overhead of embedding the query and searching the FAISS index (retrieval time), and the increased generation time required for the model to process the additional context injected into the prompt.

While this latency is a trade-off, the significant gain in factual accuracy—particularly for specific domain queries—justifies the cost for non-real-time applications. Error review shows misses split between empty recalls (threshold too strict) and skipped math steps when the LLM forgets to call the calculator. Loosening score cutoffs, forcing calculator use on numeric intents, and caching frequent topics should raise accuracy. Latency is dominated by embedding + FAISS lookup and longer generations; trimming stitched context and reusing cached contexts for follow-ups can cut several seconds without hurting correctness.

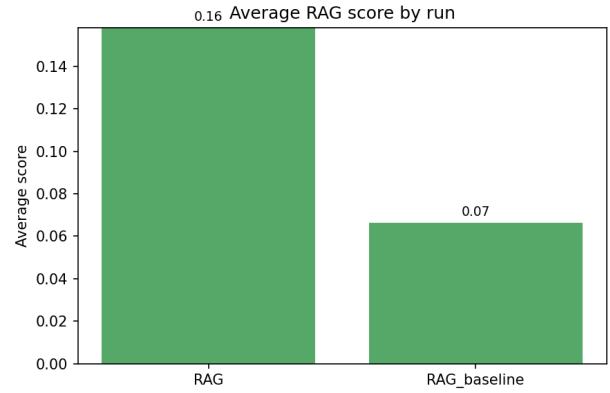


Fig. 4. Mean scores: RAG vs. baseline.

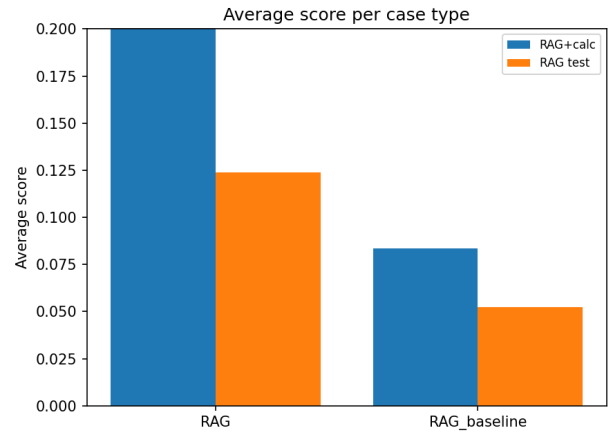


Fig. 5. Mean scores across RAG subsets (RAG+calc, RAG test).

**C. Web Search Results.** `benchmark_results/web` vs. `web_baseline`: mean 0.48 vs. 0.32 (+0.16), with latency 28.8s vs. 14.0s. Live search boosts logic and factuality but cuts throughput; low scores arise from missed searches, source mismatch, or weak aggregation. Common failures include stopping after a single search page, trusting one noisy source, or summarizing without time-aware qualifiers. Prompting for two sources before Final Answer, parallelizing fetch + summarization, and enforcing citation snippets should lift factuality. Response-length control and stricter hallucination reminders can further close the gap without increasing latency too much.



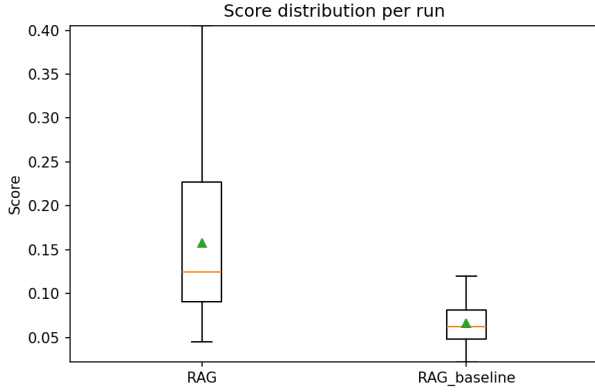


Fig. 6. Score distribution: RAG vs. baseline.

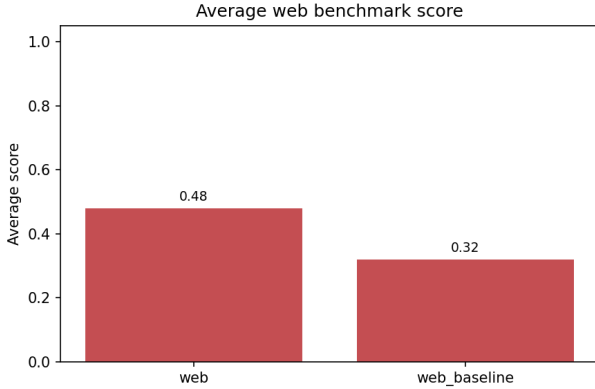


Fig. 7. Mean scores: Web search vs. baseline.

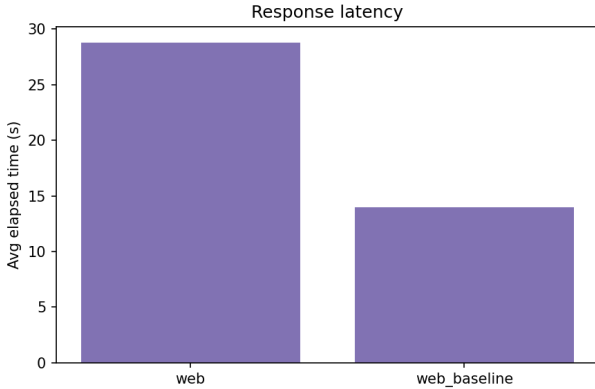


Fig. 8. Latency: Web search vs. baseline.

**D. Safety Results.** On an expanded attack set, PromptGuard detects 8/10 (20% miss) and 0/6 false positives on benign prompts. Explicit injections are blocked; encoded or oblique attacks still slip through, motivating semantic or secondary LLM judges. Misses concentrate on Unicode-obfuscated or instruction-sandwich attacks; adding homoglyph decoding and embedding-based similarity to a banned-policy bank should close recall gaps. We keep zero false positives on our benign set, but need broader multilingual and tool-use edge cases to ensure tightened rules do not over-block legitimate requests.

## 5. Ablation Design and Findings

**A. With/Without ReAct Loop.** On the same 20 ALFWorld tasks, enabling ReAct vs. disabling it yields +37.3 pct success (77.27% vs. 40.00%), +20 pct action success (83.03% vs. 63.00%), and fewer steps (17.95 vs. 27.10). Gains come from explicit Thought logging and Observation feedback; the baseline lacks planning and gets lost in branching environments. ReAct failures stem from decoder truncation; baseline failures cluster on examine+clean tasks.

**B. With/Without Retrieval (RAG Ablation).** Enabling `wiki_vector_db` vs. disabling RAG: mean 0.16 vs. 0.07, median 0.12 vs. 0.06; RAG+calc +0.12, RAG test +0.07. Cost: 11s extra latency. Retrieval context lifts long-tail and computational QA; shortening generations and caching can reduce delay.

**C. With/Without Web Search.** Disabling Web search (LLM/local only) vs. enabling Google/Tavily: mean 0.48 vs. 0.32 (+0.16) but latency 28.8s vs. 14.0s. Live search boosts timeliness and factuality but halves throughput; failures come from incomplete searches or weak citation. Parallel retrieval and summarization could cut latency.

## 6. Discussion and Future Work

Strengths: the system stays modular with interchangeable LLMs, tools, and safety modules, while the multi-modal ReAct loop naturally orchestrates tool calls and image reasoning; unified schemas across CLI/Web/API reduce drift; and the timeout/retry/circuit-breaker layer keeps long-horizon runs stable. Benchmarks and tests provide measurable regressions on embodied, RAG, Web, and safety tracks, and the logging/persistence path lets us replay Thought/Action/Observation triples to study errors. Limitations appear in latency (RAG/Web roughly 2x baseline), decoder truncation in long ALFWorld runs, and safety gaps on obfuscated attacks. Future work will focus on several key areas to address current limitations: (1) Robustness via Voting: Implementing self-consistency or multi-trajectory voting mechanisms to smooth out stochastic errors observed in ALFWorld and Web QA tasks. (2) Hallucination Mitigation: Developing tighter hallucination control by coupling retrieval confidence scores with citation granularity. The model should be instructed to refuse answering when retrieval scores fall below a strict threshold. (3) Latency Optimization: Exploring parallel retrieval paths and caching mechanisms for frequent queries to reclaim the latency costs introduced by the RAG and Web search modules. (4) Context Management: Implementing adaptive trajectory truncation and on-the-fly summarization of older thoughts to prevent decoder prompt

limits during long-horizon tasks. (5) Advanced Safety: Moving beyond regex-based guards to employ semantic safety judges or lightweight embedding filters. These would help detect obfuscated injection attacks that currently bypass heuristic checks without increasing the false-positive rate.

## 7. Conclusion

We deliver a multimodal ReAct agent that runs across CLI, FastAPI, and Web UI with reproducible ALFWorld, RAG, and Web benchmarks under course-level compute. ReAct planning yields +37 pct embodied success and +20 pct action accuracy; RAG raises mean score by 0.09 (bigger on RAG+calc) but roughly doubles latency; live search adds +0.16 mean while also doubling latency; PromptGuard blocks 80% of explicit attacks with zero benign blocks but misses some obfuscated cases. The system demonstrates that a compact, modular stack can effectively combine multimodal ReAct reasoning, retrieval capabilities, and safety layers to produce practical gains in agentic performance. By strictly adhering to modular design principles and unified interfaces, we proved that advanced AI agent architectures could be successfully implemented and benchmarked even under constrained engineering resources. Next steps center on latency cuts (parallel retrieval, caching, shorter contexts), stronger semantic safety (judge models or embedding filters), and broader regression suites to guard Web/RAG/safety behavior as components are swapped.